PV 操作 · 考点精讲与真题整合

408 操作系统大题专项 | 3 小时网课精讲 + 9 道统考真题 + 王道教材整合

这份资料整合了三个来源：① B 站《408 各科大题知识点强化课》第 12 讲「10PV操作」的完整讲稿（3 小时 11 分，本地 Whisper 转写）；② 你整理的《10PV操作》真题笔记（9 道统考真题）；③ 《王道 2027 操作系统考研复习指导》第 2 章同步互斥部分。

读法建议：先背 §3 的三张口诀卡，再看 §4 的模板，最后用 §5 的真题自测。§6 是老师反复强调的踩坑点，考前必看。

目录

1 考情分析：这个考点值多少分

2 核心概念：从并发到信号量

3 解题方法论：三张口诀卡

4 经典模型与模板

5 统考真题精解（9 道）

6 易错点清单

7 考场速查卡

1 考情分析：这个考点值多少分

1.1 分值结构

项目	分值	说明
操作系统总分	**35 分**	408 四科之一
选择题	20 分	10 题 × 2 分
大题	15 分	7 分 + 8 分两题
单题分值	7～8 分	低于数据结构 / 组成原理的 10～13 分

> 「操作系统的大题分值要少一点，但相对来说操作系统的大题还是要简单一点的，比起组成原理或者说数据结构，很多题目要简单一些。」「分数虽然少一点，但是这个分数也应该尽量拿到。」

1.2 操作系统大题的分布（老师统计）

考点	历年大题数	备注
PV 操作 / 同步互斥	**8 道**	最高频，且与「进程同步」小节内容关联
文件的物理结构	6 道	
磁盘	3 道	
其余零散考点	2 道	

> **口径提醒**
>
> 讲稿原话是「PV 操作是最重要的十一道」，与紧接着说出的「八道」在句法上接不上，疑为 ASR 误听。上表的频次按老师明确说出的「八道 / 六道 / 三道 / 零星两道」记录，**「十一道」这个数字请勿引用。**

1.3 章节内的权重与复习优先级

- **记录型信号量占 70%～80%**（老师原话：「基本考试的话这个东西能占到 70%、80%，剩下的加起来可能占到 20%」）。
- 整型信号量：「这边就只有一道大题」，权重低。
- 临界区互斥的软 / 硬件方法：「虽然也考，但是没有那么重要，我今天的话放到最后来讲」——**本讲未展开。**

> **老师给的复习时间表**
>
> 「目前整体只考过生产者消费者（和）哲学家进餐，现阶段把这两个学透就足够。**到 11 月份时间充裕可以把附上的读者写者看一看；如果 11 月时间不充裕，读者写者就不用管。**」

> 「408 的选择题考的很散，但是大题考的往往比较集中。」「在当前这个阶段一定要着重的去练习这个大题。」「学习知识点的时候一定要分清主次，就是先把那种考评最高的先学了。」

2 核心概念：从并发到信号量

2.1 五个前置概念

概念	定义（老师讲法）
并发	两个事件在同一时间间隔内发生。
异步	「这个进程以不可预知的速度推进，这个叫异步。」几个程序「有可能按照一二三四去执行，有可能按照四三二一」「他们是来回交替的」「甚至一执行一半又去执行到三」。
同步	「你要满足你想要的顺序，按照你想要的顺序去执行叫同步。」异步和同步是反义词。
互斥	两进程抢一台打印机：「第一个进程进到临界区使用临界资源，另一个进程去等待，只有前面那个退出临界区，另一个进程才能去用。」
临界资源	「一次只让一个进程使用的资源叫临界资源。」
临界区	「访问临界资源的代码叫临界区。」

「后面要学的 PV 操作主要就是解决这样的一个问题。」

2.2 记录型信号量：考试的主体

采用结构化数据，除整型字段 `value`（当前可用资源数）外，还有一个进程链表 `L`（等待队列）。老师：「这个 L 是一个链表，你也可以理解为是一个队列，本质上它是用链表实现的一个队列，就把它当成队列去做就可以。」

```
typedef struct {
    int value;              // 当前可用资源数
    struct process *L;      // 等待队列（阻塞在该信号量上的进程）
} semaphore;
```

P 操作（wait）与 V 操作（signal）

```
void wait(semaphore S) {        // P 操作：申请资源
    S.value--;
    if (S.value < 0) {          // 注意是 < 0
        add this process to S.L;
        block(S.L);             // 自我阻塞，让出 CPU
    }
}

void signal(semaphore S) {      // V 操作：释放资源
    S.value++;
    if (S.value <= 0) {         // 注意是 <= 0
        remove a process P from S.L;
        wakeup(P);              // 唤醒一个等待进程
    }
}
```

本讲最值得背的一句

S.value > 0 时，表示当前还可用的资源数；S.value < 0 时，其绝对值等于正在等待（被阻塞）的进程个数。

老师原话：「就是这个信号量如果是负数，它的绝对值刚好就是阻塞的进程 [个数] 。」

为什么要原语

「PV 操作它是一个标准的原语，执行 PV 操作的时候不能被打断的，原语就是这个不可分割的最小的单位。」若对共享变量的操作被中途打断，可能与其他进程的操作交织，破坏临界区互斥性。

整型信号量为什么被淘汰：它的 wait 是 `while (S<=0);` 的忙等待，不遵循「让权等待」准则，浪费 CPU。记录型信号量通过阻塞机制实现了让权等待。

教材标注的考点年份

信号量的含义（2010）· wait() 操作导致线程状态的变化（2023）· 遵循「让权等待」的互斥方法（2018）· 利用信号量实现互斥的实现（2024）· PV 操作的应用题（2009、2011、2013、2014、2015、2017、2019、2025）

3 解题方法论：三张口诀卡

这一节是全篇最该背下来的部分。老师反复强调：**判分看两个点——① 信号量定义了几个、初值是多少；② P、V 放的位置。**其余都是形式。

3.1 三张口诀卡

关系类型	信号量初值	P / V 摆法
互斥	1	临界区**前** P、临界区**后** V，把临界区包住。老师：「定义一个互斥的话一般是让它的初值置为 1」「用 PV 去包住这个互斥的东西就可以了」。
同步	0	**先发生的后面加 V，后发生的前面加 P。**
前驱图	0（每条边一个信号量）	同同步。老师：「你只需要在每一个边上定义一个信号量」「由于前驱关系本质上就是同步关系，所以它会把每一个信号量初值都置为 0」。

最容易记反的一句

老师在前驱图段落先说错了一次（「先发生的前面加 P，后发生的后面加 V」），随即当场更正为：**「先发生的是后面加 V，然后后发生的前面加上一个 P。」**做题只用更正版。

老师的三句关键原话

① 「你定义信号量的数量是一个很关键的点，一个是数量，一个是初值，很关键。」
② **「互斥关系初值都是一。你如果初值设置的不是一，它其实就是一个同步关系。」**
③ **「先发生的后面加 V，后发生的前面加 P。」**另有：「出现这种『等』的，一定要把它换成 P 操作。」

3.2 怎么判断是互斥还是同步

- 无先后要求、一次只能一个进程用 → **互斥**，初值 1。
- 有明确的先后要求 → **同步**，初值 0。
- **生产者—消费者两者都有。**老师：「生产者消费者明显既有互斥也有同步。比如说生产者消费者，你往缓冲区放东西是互斥的；同步的话就是说你必须有空位才能往里面放，放满之后才能往外拿。」
- 题目措辞不可全信：「他说实现同步，这个信箱……生产者消费者明显既有互斥也有同步」「习惯上的同步和互斥你只能作为参考，并不能完全依赖」。

3.3 答题六步（教材三步法的可操作版）

1. **画图。**「像这种题目一定要把图画出来」——把约束画成前驱图或资源流向图。

2. **数关系、定信号量个数。**前驱图每条边一个；同步一个先后关系一个；互斥一个临界资源一个。**同一线程内部的天然顺序不要再定义信号量。**
3. **写初值。**互斥 1；同步 0；资源计数按题给初始量（如空位数 `M-x`、已有数 `x`）。
4. **摆 P / V。**先发生的后面 V，后发生的前面 P。
5. **补形式。**while 循环、花括号、分号、注释。老师：「这些弄好之后，其他的都是一些形式上的东西。」
6. **若题目问「说明信号量含义」就逐个写清**含义与初值。

> 「考试的话这个**初值往往是一个改卷比较关注的点**，一定要回答正确。」「第一个是信号量定义了多少个，以及初值是多少，这是一个比较看重的点；第二个就是 P、V 放的位置。」

3.4 原理推演（理解用，不用写在卷面上）

互斥为什么成立（mutex 初值 1，P1/P2/P3 争临界区）：

```
P2 的 P:  1 → 0    不小于 0，进入临界区
P3 的 P:  0 → −1   < 0，阻塞，进等待队列
P1 的 P: −1 → −2   < 0，阻塞
P2 的 V: −2 → −1   ≤ 0，唤醒一个（P3）
P3 的 V: −1 → 0    ≤ 0，唤醒一个（P1）
P1 的 V:  0 → 1    不满足 ≤ 0，队列已空，无需唤醒
```

同步为什么成立（初值 0）：不该先跑的进程先跑了，P 操作把 0 减成 -1 而阻塞；该先跑的进程跑完后 V 操作把 -1 加回 0 并唤醒它，顺序就被强制保证。

> 「真正在做的时候，其实并不需要写这些东西，直接 P、V 就行了……但是你得知道它底层的原理」「不能停留在只会用」。

4 经典模型与模板

4.1 生产者—消费者（必须能默写）

「PV 操作考察的频率最高的」模型。三信号量：

```
semaphore mutex = 1;     // 互斥访问缓冲区
semaphore empty = n;     // 空缓冲区数量（初值 = 容量）
semaphore full  = 0;     // 满缓冲区数量（初值 = 0，初始为空）

producer() {                          consumer() {
    while (1) {                           while (1) {
        生产一个产品;                          P(full);        // 要用什么，P 一下
        P(empty);   // 要用什么，P 一下        P(mutex);       // 互斥夹紧
        P(mutex);   // 互斥夹紧               从缓冲区取产品;
        将产品放入缓冲区;                      V(mutex);       // 互斥夹紧
        V(mutex);   // 互斥夹紧                V(empty);       // 提供什么，V 一下
        V(full);    // 提供什么，V 一下        消费产品;
    }                                     }
}                                     }
```

铁律：P 的顺序不能反

`empty` / `full` 的 P 必须在 `mutex` 的 P 之前。

反写的后果（原话）：「假设你先执行一个 P(mutex) 相当于加了把锁，然后再往里面放，因为现在已经没有空位置了，你执行 P(empty) 就会阻塞。现在就相当于**生产者明明放不进去，但是他会占着 mutex 不释放**。然后理论上现在是可以往外拿的，但是消费者去拿的时候……会发现 mutex 这个锁还没有释放……所以一旦你把这个顺序写反是一个很严重的问题，会造成死锁。」「就整个程序就停了……就僵持住。」

mutex 的三层含义（老师要求理解到这个地步）

① 生产者之间互斥 ② 消费者之间互斥 ③ 生产者与消费者之间互斥。

「这段代码你可以想象成好几个进程都在执行这段代码，它不是一个进程。」

加锁比喻：「你 P(mutex) 就可以理解为加锁，V 的话相当于你把锁打开。就好比有一个房间，有一个人进门之后把门反锁了，那肯定其他人进不去了，直到这个人把门打开之后，其他人才能进。」

关于 while 循环：「很多题目会加上这个 while 循环……他不是很关键的得分点，就是说写起来让代码变得更美观，这个只是锦上添花的作用。」

4.2 前驱图 → PV 序列

方法：**每一条边定义一个信号量，初值全 0；边的起点后面 V，边的终点前面 P。**

例：S1–S6 的 7 条边（1→2, 1→3, 2→4, 2→5, 3→6, 4→6, 5→6）：

```
S1: ...;                V(a12); V(a13);
S2: P(a12); ...;        V(a24); V(a25);
S3: P(a13); ...;        V(a36);
S4: P(a24); ...;        V(a46);
S5: P(a25); ...;        V(a56);
S6: P(a36); P(a46); P(a56); ...;
```

例外：线程内部的天然顺序不要写

「有的人在做的时候可能会上来就把每一个边定义信号量，**不要这样做**。」

「因为 AEF 本来就在一个线程里，所以每一个线程他自己的代码本来就是串行执行的」「A 本来就在 E 前面，E 本来就在 F 前面，**这个东西是不需要你去写同步的，一定要注意这个**。」

所以 2022 真题（T1 执行 AEF、T2 执行 BCD）最终**只剩两条跨线程的边，只需两个信号量**。

4.3 读者—写者

读者优先（可能写者饥饿）：

```
int count = 0;                  // 当前读者数量
semaphore mutex = 1;            // 互斥访问 count
semaphore rw = 1;               // 读者与写者互斥访问文件

reader() {                          writer() {
    while (1) {                         while (1) {
        P(mutex);                           P(rw);
        if (count == 0) P(rw);  // 首个读者挡住写者    写文件;
        count++;                            V(rw);
        V(mutex);                       }
        读文件;                     }
        P(mutex);
        count--;
        if (count == 0) V(rw);  // 末个读者放开写者
        V(mutex);
    }
}
```

写者优先：再引入互斥信号量 `w` 作为「门卫」。写者先 `P(w)` 再 `P(rw)`；读者也先 `P(w)`，更新完 count 后立即 `V(w)`——只要还有活跃读者，写者已被 `rw` 挡住，提前放 w 可避免不必要地阻挡其他写者。这样一旦有写者等待，新读者就无法进入。

现阶段要不要看

老师明确：读者—写者**本次课没讲**，且「到 11 月份时间充裕可以把读者写者看一看；如果 11 月时间不充裕，读者写者就不用管」。

4.4 哲学家进餐（两种解法都要会）

```
// 解法一：mutex 限制每次只让一个人拿筷子（不死锁，但并发度低）
semaphore chopstick[5] = {1,1,1,1,1};
semaphore mutex = 1;
哲学家 i:
    P(mutex);                       // 拿筷子前先互斥
    P(chopstick[i]);
    P(chopstick[(i+1) % 5]);
    V(mutex);
    eat();
    V(chopstick[i]);
    V(chopstick[(i+1) % 5]);
    think();
```

```
// 解法二：用计数信号量限制同时进餐的人数（至多 4 人）
semaphore chopstick[5] = {1,1,1,1,1};
semaphore s = 4;                    // 至多 4 人同时竞争筷子
哲学家 i:
    P(s);
    P(chopstick[i]);
    P(chopstick[(i+1) % 5]);
    eat();
    V(chopstick[i]);
    V(chopstick[(i+1) % 5]);
    V(s);
    think();
```

老师纠正教材的一处表述

教材写「当左右筷子都可用时才允许抓筷子」，老师说**这个描述不对**——上述 mutex 解法的实际意思是「**每次只让一个人拿筷子**」，而不是「检查两根都空闲才拿」。做题按后者理解会被带偏。

4.5 两个通用技巧

技巧一：拆分信号量

当一个信号量要表达两种不同含义时，必须拆开。

· **奇偶统计题**：单个 `full` 无法区分奇偶，拆成 `odd` / `even`。

· **多线程共享多变量题**：一个信号量管多个变量会造成过度互斥（本可并行的被串行化），把 `y` 拆成 `y1` / `y2`。

技巧二：成组操作加外层锁

要求「连续做 N 次」时，在整组外面再加一个互斥量，但**计数信号量的 P 必须留在循环内部**。

5 统考真题精解（9 道）

以下 9 道题取自你整理的《10PV操作》真题笔记，题面经过 OCR 校正与语句理顺，解法保留原笔记的信号量设置与 PV 序列。

2020 五操作前驱关系

题面：现有 5 个操作 A、B、C、D 和 E。操作 C 必须在 A 和 B 完成后执行，操作 E 必须在 C 和 D 完成后执行。请使用信号量的 wait()、signal() 操作描述上述操作之间的同步关系，并说明所用信号量及其初值。

分析：前驱图，四条边 → 四个信号量，初值全 0。

```
semaphore a = 0, b = 0, c = 0, d = 0;

A: ...;              V(a);
B: ...;              V(b);
C: P(a); P(b); ...;  V(c);
D: ...;              V(d);
E: P(c); P(d); ...;
```

> 老师验证
>
> 「假如说现在 C 和 D 还没有执行，你上来就去执行 E，那它执行 Pc 的时候 C 本来是 0，你 Pc 一下就变成 -1 了，那就阻塞了……只有 C 和 D 都执行完之后，你这边的 Pc、Pd 才不会被阻塞。」

2022 双线程同步（线程内部顺序不写信号量）

题面：某进程的两个线程 T1 和 T2 并发执行 A、B、C、D、E 和 F 共 6 个操作，其中 T1 执行 A、E 和 F，T2 执行 B、C 和 D。执行顺序须满足：C 在 A 和 B 完成后执行，D 和 E 在 C 完成后执行，F 在 E 完成后执行。请用 wait()、signal() 描述 T1 与 T2 之间的同步关系，并说明信号量的作用及初值。

> **本题唯一的坑**
>
> 不要机械地「每条边一个信号量」。**A→E→F 是同一线程内的天然顺序，B→C→D 同理，都不需要写同步。**只剩 A→C、C→E 两条跨线程边，两个信号量足矣。

```
semaphore S_AC = 0;     // A 完成后 C 才能执行
semaphore S_CE = 0;     // C 完成后 E 才能执行

// 线程 T1:  A; V(S_AC);  ... E: P(S_CE); ...; F;
// 线程 T2:  B; C: P(S_AC); ...; V(S_CE); D;
```

2010 信箱辩论（生产者—消费者的包装）

题面：A、B 两人通过信箱辩论，每人都从自己的信箱中取得对方的问题，将答案和向对方提出的新问题组成邮件放入对方的信箱。A 的信箱最多放 M 个邮件，B 的信箱最多放 N 个。初始时 A 的信箱有 x 个邮件（0<x<M），B 的信箱有 y 个（0<y<N）。当信箱不为空时才能取，不满时才能放。请添加必要的信号量和 P、V 操作实现同步，写出完整过程并说明信号量含义和初值。

> **两个坑**
>
> ① **变量要两组**：「对 A 信箱的操作是互斥的，对 B 信箱的操作也是互斥的，你肯定不能通过一个变量去实现。」
>
> ② **初值不是容量而是剩余空间**：A 能放 M 格、已有 x，空位是 `M−x`；B 是 `N−y`。老师：「第一个初值是 X，第二个初值是 Y」（指 fullA / fullB）。

```
semaphore mutexA = 1, mutexB = 1;
semaphore emptyA = M − x, emptyB = N − y;
semaphore fullA  = x,     fullB  = y;

A: P(fullA);  P(mutexA); 从 A 信箱取邮件;  V(mutexA); V(emptyA);
   回答问题并提出新问题;
   P(emptyB); P(mutexB); 将新邮件放入 B 信箱; V(mutexB); V(fullB);
B: 与 A 对称（把 A/B 互换）
```

2009 三进程奇偶统计（把 full 拆成两个）

题面：三个进程 P1、P2、P3 互斥使用一个包含 N（N>0）个单元的缓冲区。P1 每次用 produce() 生成一个正整数并用 put() 送入缓冲区某一空单元；P2 每次用 getodd() 取出一个奇数并用 countodd() 统计奇数个数；P3 每次用 geteven() 取出一个偶数并用 counteven() 统计偶数个数。请用信号量机制实现这三个进程的同步与互斥，并说明所定义信号量的含义（用伪代码描述）。

> **关键改动**
>
> 「如果你单纯定义一个 full 变量，它是无法区分奇数和偶数的，这个 full 变量只是用来统计缓冲区中元素的个数。所以这个题目要把这个 full 变量**拆成两个**。」

```
semaphore mutex = 1, empty = n, odd = 0, even = 0;

P1: while(1) { x = produce();
               P(empty); P(mutex); put(x); V(mutex);
               if (x % 2 == 1) V(odd); else V(even); }

P2: while(1) { P(odd);  P(mutex); getodd();  V(mutex); V(empty); countodd();  }

P3: while(1) { P(even); P(mutex); geteven(); V(mutex); V(empty); counteven(); }
```

老师的三点说明

① 「单纯这个生产的过程其实并不需要去互斥的，生产完之后你把这个元素放到缓冲区是互斥的，就这个『放』是互斥的。」

② 「统计这个就不需要互斥了，只是说取的那一下需要互斥。」

③ 「取奇数和取偶数他们两个没区别，直接复制三行就行，代码是一模一样的，无非把这个变量名变一下。」

2014 环形缓冲区 + 消费者连续取 10 件（错误写法最多的题）

题面：多个生产者进程和多个消费者进程共享一个能存放 1000 件产品的环形缓冲区（初始为空）。缓冲区未满时生产者可放入，未空时消费者可取走。要求**一个消费者进程从缓冲区连续取出 10 件产品后，其他消费者进程才可以取产品**。请用 P、V 操作实现互斥与同步，写出完整过程并说明信号量含义和初值。

```
semaphore mutex1 = 1;       // 生产者/消费者对缓冲区的互斥
semaphore mutex2 = 1;       // 消费者之间「连续取 10 件」的互斥
semaphore empty  = 1000, full = 0;
int in = 0, out = 0;

生产者: while(1) {
    P(empty);  P(mutex1);
    放入产品;  in = (in + 1) % 1000;
    V(mutex1); V(full);
}

消费者: while(1) {
    P(mutex2);                          // 外层锁：包住整组
    for (int i = 1; i <= 10; i++) {
        P(full);                        // ← 必须留在循环内部
        P(mutex1);
        取出产品;  out = (out + 1) % 1000;
        V(mutex1); V(empty);
    }
    V(mutex2);
}
```

错误写法	为什么错
① 只用一个互斥量	「可以理解为这段代码是多个消费者同时执行……很有可能第一个消费者能执行这个 for 循环，第二个消费者也能去执行，其实你这样写是满足不了题目的要求的。」
② 把 `P(mutex1)` 放在 for 循环外面	(a) **死锁**：缓冲区空时消费者先锁上 mutex1，再 P(full) 阻塞，生产者有空位却拿不到锁；(b) **粒度错**：这十次里生产者本可插入（题目并未禁止），却被强制互斥掉。
③ 把 `P(full)` 放在 for 循环外面	只 P 一次只保证「至少有 1 个」，不保证「有 10 个」。若缓冲区只有 3 个产品，循环取到第 4 次就取不出来了。

老师的提醒（关于本题答案）

「这个 20 题**不同的人可能答案不一样**……我的电子版书上反正是有点问题，到时候可以留一下自己的纸质的那个书的答案。」「这个有的数字可能他没有考虑到环形缓冲区……写的时候要注意下有没有考虑到这个东西。」

→ 也就是说：**环形缓冲区的 `in=(in+1)%N`、`out=(out+1)%N` 取模不能漏**，做这题以纸质书答案为准。

2013 博物馆限流（只有生产者，没有消费者）

题面：某博物馆最多可容纳 500 人同时参观，有一个出入口，该出入口一次仅允许一人通过。参观者的活动描述如下：`cobegin 参观者进程 i: … coend`。请添加必要的信号量和 P、V 操作实现互斥与同步，写出完整过程。

```
semaphore mutex = 1;        // 出入口互斥
semaphore empty = 500;      // 馆内剩余容量

参观者进程 i:
    P(empty);               // 馆内没满才能进
    进门;
    参观;
    出门;
    V(empty);               // 离馆释放容量
```

本题的辨识点

「只有生产者没有消费者，full 用不上。」不要看到「容量」就条件反射地写 empty/full 一对——这里没有取走资源的一方，只需要一个计数信号量。

`mutex` 在此承担三层含义：进门互斥、出门互斥、一进一出之间的互斥。

2011 银行叫号（四个信号量）

题面：某银行提供 1 个服务窗口和 10 个供顾客等待的座位。顾客到达银行时，若有空座位，则到取号机上领取一个号，等待叫号。取号机每次仅允许一位顾客使用。当营业员空闲时，通过叫号选取一位顾客并为其服务。请添加必要的信号量和 P、V 操作实现互斥与同步，写出完整过程，说明信号量含义并赋初值。

```
semaphore empty = 10;   // 空座位数
semaphore mutex = 1;    // 取号机互斥
semaphore full  = 0;    // 已取号等待的顾客数
semaphore s     = 0;    // 营业员是否空闲（叫号同步）

顾客 i:                          营业员:
    P(empty);                        while (1) {
    P(mutex);                            P(full);    // 有人等着才叫号
    从取号机获取一个号码;                  叫号;
    V(mutex);                            V(s);
    V(full);                             等待顾客到位并服务;
    P(s);                            }
    获取服务;
    V(empty);
```

三个易错点

① **「等」必须写成 P，不能写中文。**老师：「出现这种『等』的，一定要把它换成 P 操作。」

② **顾客与营业员不互斥**——「千万不要在叫号处加 mutex」。

③ `s` 是最难想到的那个信号量：它表达「营业员叫完号、顾客才能去接受服务」这个同步点。

年份不详 三线程共享复数变量（拆分信号量）

题面：有如下全局定义，三个线程并发执行，要求确保线程互斥访问临界资源，**并且最大限度地并发执行。**

```
typedef struct { float a; float b; } cnum;
cnum x, y, z;                    // 全局变量
cnum add(cnum p, cnum q) { cnum s; s.a = p.a + q.a; s.b = p.b + q.b; return s; }

thread1: { cnum w; w.a = 1; w.b = 1; x = add(x, w); y = add(y, w); }
thread2: { cnum w; w = add(z, w); }   // 与 thread3 协作
thread3: { … }
```

解题关键：拆分信号量

若只用一个信号量同时管 `x` 和 `y`，会把本可并行的两个赋值强行串行化，违背「最大限度并发」的要求。**把 `y` 拆成 `y1` / `y2`**（分别对应两个线程对 y 的使用），即可在保证互斥的前提下恢复并发度。

2019 哲学家进餐（n 人 / m 碗 / 每两人一根筷子）

题面：有 n（n≥3）名哲学家围坐圆桌，每名哲学家交替地就餐和思考。圆桌中心有 m（m≥1）个碗，每两名哲学家之间有一根筷子。每名哲学家必须取到一个碗和两侧的筷子后才能就餐，进餐完毕将碗和筷子放回原位。为使尽可能多的哲学家同时就餐，且防止死锁，请用 P、V 操作描述互斥与同步，并说明信号量及初值的含义。

核心结论

`semaphore S = min(m, n-1);` 为什么是 `n-1`：至多允许 n-1 人同时抢筷子，必然有人能拿齐两根，从而破坏循环等待。

```
// 解法一：用计数信号量 S 限制同时拿筷子的人数
semaphore S = min(m, n - 1);
semaphore chopstick[n];
for (int i = 0; i < n; i++) chopstick[i] = 1;   // 必须用循环赋初值

哲学家 i:
    while (1) {
        P(S);
        P(chopstick[i]);
        P(chopstick[(i + 1) % n]);
        eat();
        V(chopstick[i]);
        V(chopstick[(i + 1) % n]);
        V(S);
        think();
    }
```

```
// 解法二：碗与筷子分别管理
semaphore bowl = m, mutex = 1;
semaphore chopstick[n];
for (int i = 0; i < n; i++) chopstick[i] = 1;

哲学家 i:
    while (1) {
        P(bowl);
        P(mutex);                       // 拿筷子这个动作本身互斥
        P(chopstick[i]);
        P(chopstick[(i + 1) % n]);
        V(mutex);
        eat();
        V(chopstick[i]);
        V(chopstick[(i + 1) % n]);
        V(bowl);
        think();
    }
```

形式分

「N 个信号量数组**必须用 for 循环赋初值**」——直接写 `chopstick[n] = 1` 这种写法语法上不成立，是常见扣分点。

6 易错点清单

#	易错点	说明
1	**P 的顺序写反 → 死锁**	资源信号量（empty/full）的 P 必须在 mutex 的 P 之前。老师：「一旦你把这个顺序写反是一个很严重的问题，会造成死锁。」
2	**初值写错**	「考试的话这个初值往往是一个改卷比较关注的点，一定要回答正确。」互斥=1，同步=0，资源计数按题给初始剩余量。
3	**机械地「每条边一个信号量」**	同一线程内部的天然顺序不需要写同步——2022 真题的关键区分点。
4	**该拆的信号量没拆**	一个信号量表达两种含义时必须拆：full→odd/even；y→y1/y2。
5	**计数信号量的 P 放到循环外**	2014 真题错误写法③：只 P 一次只保证「至少 1 个」，不保证「有 10 个」。
6	**看到「容量」就写 empty/full 一对**	2013 博物馆题只有生产者没有消费者，`full` 用不上。
7	**把题目里的「等」照抄成中文**	「出现这种『等』的，一定要把它换成 P 操作。」
8	**在不该互斥的地方加 mutex**	2011 题顾客与营业员不互斥，叫号处不要加 mutex。统计类操作也不需要互斥，只有「取/放」那一下需要。
9	**信号量数组不写循环赋初值**	2019 题的形式分。
10	**忘写 while 循环**	「不是很关键的得分点」，但「一定要注意观察一下」。
11	**照抄题目措辞判断互斥/同步**	「习惯上的同步和互斥你只能作为参考，并不能完全依赖。」2010 信箱题说「实现同步」，实际两者都有。
12	**死记中间值的推演**	「熟练之后那些信号量怎么变的都不用管了」——理解原理即可，卷面只写 PV。
13	**背答案**	「也不要背答案，就是要完全理解，要能理解答案的每一句为什么要这样写。」
14	**环形缓冲区漏取模**	`in=(in+1)%N`、`out=(out+1)%N` 不能漏；老师指出其电子版教材此处「完全没考虑」。

7 考场速查卡

> **口诀**
>
> **互斥：初值 1，P 在临界区前、V 在临界区后。**
> **同步：初值 0，先发生的后面加 V，后发生的前面加 P。**
> **前驱图：每条边一个信号量，初值全 0；线程内部天然顺序不写。**
> **资源计数：empty = 容量（或 M−x），full = 已有数（或 x）。**

```
生产者–消费者模板（必须能默写）
    P(empty) → P(mutex) → 放 → V(mutex) → V(full)
    P(full)  → P(mutex) → 取 → V(mutex) → V(empty)
    铁律：empty / full 的 P 必须在 mutex 的 P 之前，写反 = 死锁

成组操作（连续取 10 个）：再加一个 mutex 包住整组
    P(mutex2) → for(10次){ P(full); P(mutex1); 取; V(mutex1); V(empty); } → V(mutex2)
    注意：计数信号量的 P 必须在循环里面；外层 mutex 不能替代内层 mutex
```

得分点

① 信号量定义了几个 + 初值是多少　② P、V 摆的位置　③ 其余（分号、花括号、注释、while）都是形式。

老师的练习建议

「你把同一类型的大题连着做四五道，我觉得这一块基本就理解透彻了」「生产者消费者 PV 操作这块基本是一通百通的……像什么读者写者、哲学家进餐，都一样的，道理都是相通的」「绝大多数真题的 PV 操作都是能够写出跟答案一模一样的」。

附：本资料的来源与边界

来源	内容	可靠性与边界
B 站《408 各科大题知识点强化课》第 12 讲（BV1hQy8YdE1Z，分 P12，3 小时 11 分）	§1 考情、§2 概念、§3 方法论、§4 模板、§6 易错点	本地 Whisper 转写，ASR 同音错误较多（mutex 曾被识别为 multax/multiplex，信号量为「忽视量/新花量」等），引用原话均已结合上下文校正。讲稿**无板书**，所有伪代码是依据老师逐句口述的命令顺序还原的。
你的《10PV操作_整理版.pdf》（11 页）	§5 全部 9 道真题	该 PDF 嵌入字体缺 ToUnicode 表，文本层提不出中文，已用 macOS Vision 在本机 OCR 后人工校正题面。
《王道 2027 操作系统考研复习指导》第 2 章	§2.2 记录型信号量定义、§3.3 三步法、§4.1/4.3 代码	教材为扫描件，本资料引用的页码段落来自对全书 372 页的本地 OCR。教材代码与老师口述版本在个别变量名上略有差异，语义一致。

三处需要你自己核对的

① **2014 题（连续取 10 件）**：老师明确说他自己手上的电子版教材答案有问题，请以纸质书答案为准。
② **2022 题题面**：原 PDF 的 OCR 在此处有错行，约束条件是按老师「最后只剩两条边」的结论反推的，请对照原题确认。
③ **复数题年份**：原笔记未标注年份，此处从缺。

本讲未覆盖的内容

临界区互斥的软件方法（单标志法、双标志先检查/后检查、Peterson 算法）与硬件方法（TS 指令、Swap 指令）——老师明确说「放到最后来讲」，本讲未展开。

读者—写者问题——老师明确说本次课不讲，建议 11 月视时间决定。
